Supported Git, Safari, and Linux kernel versions

Both floors below are derived from something the codebase actually depends on, not asserted. Where the evidence is thin, that's said explicitly rather than rounded up to a plausible-sounding number.

Git: 2.32 or later

Why 2.32: the `GIT_CONFIG_GLOBAL` environment variable — which lets git-vista's write path pin the global-config file it reads without touching `$HOME`, so it can override a user's `core.hooksPath` or credential helper deterministically rather than trusting whatever the ambient environment provides — was added in Git 2.32 (June 2021). Below that version the variable does not exist and config isolation falls back to weaker mechanisms.

This floor is tied to the git-process-execution-policy work landing in #66 (M1.13, in progress in a parallel session as of this writing), which is what actually depends on `GIT_CONFIG_GLOBAL`. That policy is not yet merged to `main`, so as of this doc nothing on `main` strictly requires 2.32 yet — the number is documented now because it's the floor the in-flight design commits to, and because current dev/CI environments already comfortably clear it (see below), so recording it early costs nothing and avoids the floor drifting undocumented once #66 lands.

- **Empirically confirmed:** this repo's dev environment and this session's test runs are on **git 2.43.0** (`git --version`), well above the 2.32 floor.
- **CI:** `ubuntu-latest` GitHub-hosted runners currently ship a git version in the same 2.4x range, also clear of the floor. **Now checked**: the `core` job's "Git version meets the documented floor" step (#67 M1.14) parses the `## Git: X.Y or later` heading above out of this file and fails the build if the runner's `git --version` is older — one source of truth, so the doc and the check cannot drift apart. Added ahead of #66 actually landing, on the same reasoning as documenting the number early: it costs nothing now and means the floor is already enforced the moment it becomes load-bearing rather than being remembered later.
- **Exercised, not only enforced (#365):** enforcing "not older than 2.32" is not the same as having run anything at 2.32, and until #365 nothing had. The `core` job now builds a real 2.32 binary and runs the whole working-tree status vocabulary through `parse_porcelain_v2_z` with both it and the runner's git, holding each to a named expected value. The floor number below is still the only place the version is written down — the job parses this heading for the tag to build, so a change here moves the test. See ADR 0082 for why the leg is mandatory and how that survives a transient fetch failure.

The finding: the floor and the current git parse identically, on every shape and under all three read modes — measured at 2.32.0 against 2.43.0 on a developer box, and 2.32.0 against **2.55.0** on the CI runner. The upper end is deliberately not pinned: it is whatever git the machine has, so the span widens on its own as runners move. That is the

expected result, and it is now measured on every run rather than inferred from git's release notes.

Reproducing the floor leg locally

`cargo test --workspace` runs the status battery against whatever git is on your `PATH`. To add the floor leg, build the floor once and point the test at it. About a minute on four cores; `msgfmt` and `tclsh` are not needed to run `git status`, so the build skips them:

```
floor=$(grep -oP '^## Git: \K[0-9]+\.[0-9]+' docs/SUPPORTED_VERSIONS.md)
src=$(mktemp -d)
git clone --depth 1 -b "v${floor}.0" https://github.com/git/git "$src/git"
make -C "$src/git" -j"$(nproc)" prefix="$HOME/.cache/gv-git-floor" \
  NO_GETTEXT=1 NO_TCLTK=1 NO_CURL=1 NO_EXPAT=1 NO_PERL=1 NO_PYTHON=1 install
```

Then, from the repository root:

```
GV_GIT_FLOOR="$HOME/.cache/gv-git-floor/bin/git" \
GV_STATUS_FLOOR_REPORT=/tmp/gv-status-floor-report.txt \
  cargo test -p git-vista-fixtures --test status_floor
```

The report names both binaries and every shape each one read. Without `GV_GIT_FLOOR` the test still runs and still checks the current git against the expectations — it records `floor=unrun`, and CI rejects that. The test never decides for itself whether the floor leg was required; that is asserted in shell over the report, per ADR 0082.

A binary that is not the documented floor is refused before it is compared against anything, so pointing `GV_GIT_FLOOR` at a second copy of your ordinary git fails rather than comparing a version with itself.

Feature floors above the product floor

The heading above is the **product** floor: everything git-vista does works at 2.32, and #365 builds and exercises a real 2.32 binary to prove it. **Two** features need more than that, and both degrade rather than raising the floor for everyone.

Feature	Needs	Why	Below it
Graph preview (M10.08, #576)	**git 2.38**	`git merge-tree --write-tree` — the plumbing that computes the real three-way merge without a worktree or an index — arrived in 2.38.0	The server starts, everything else works, and the preview alone answers `Unavailable { GitTooOld { found, minimum } }`
Revert offer (#327, corrected by #581)	**git 2.38**	the same `merge-tree --write-tree`: `activity::revert_would_conflict` uses it to establish whether reverting a commit would conflict, rather than guessing	The server starts, everything else works, and the revert offer alone is withheld — `RevertCheckError::GitTooOld { found, minimum }`, surfaced by the Recovery Center as `CheckFailedReason::GitTooOld`

#581 — the row that was missing, and what it cost. The revert row above was true from #327 onward and undocumented until #581. `revert_would_conflict` ran `--write-tree` with no version check of any kind, so on a host inside the documented 2.32–2.37 band the call failed and the revert offer simply never appeared, with no reason given to the user. It degraded fail-closed, which is exactly why it went unnoticed: the posture was right and only the explanation was missing.

Measured 2026-09-02, running the argv `revert_would_conflict` builds against two real gits in containers:

git	exit	output
2.34.1 (Ubuntu 22.04 LTS)	**129**	`usage: git merge-tree <base-tree> <branch1> <branch2>`
2.43.0 (Ubuntu 24.04 LTS)	0	the merged tree oid

129 is neither the documented 0 (clean) nor 1 (conflict), so it fell into the "the check itself did not answer" arm and stayed there. Ubuntu 22.04 LTS is the distribution that matters here: it is inside the supported band and ships 2.34.1.

Why this is a table and not a second floor. A host on 2.32–2.37 is a fully supported host. Raising the product floor to 2.38 for one feature would refuse service to a machine on which every other feature is correct, and the boot gate that would have to enforce it (`sandbox::probe`) is deliberately the one gate in this codebase with no degraded outcome at all — "a verdict other than `Contained` means no server, full stop" (ADR 0029). A capability question does not belong in a gate whose whole argument is that it has none.

The check therefore lives in the feature, not in the boot gate. Since #581 the measurement is shared and the policy is not: `crates/git-vista-server/src/git_version.rs` establishes the running git's version once per process (one probe, one parser, one comparison), and each feature keeps its own floor constant — `preview::MIN_GIT_FOR_PREVIEW` and `activity::MIN_GIT_FOR_MERGE_TREE`. Both are 2.38 today and that is a coincidence of the same plumbing, not a shared policy: folding them into one constant would quietly recreate a second product floor, which is the thing this section exists to avoid. Both are deliberately separate from the number in the heading above. Reasoning in full: **ADR 0099** (the gate) and **ADR 0106** (why the measurement is shared and the floors are not).

Do not fold this number into the `## Git:` heading. That heading is parsed by the `core` job's floor check (#67) and names the tag #365's floor test builds; changing it moves both.

Safari: 16.4 or later (iOS/iPadOS 16.4, and the matching macOS Safari 16.4)

Why 16.4: the frontend's app shell and full-screen overlays size themselves with the CSS dynamic-viewport-height unit (`dvh`), specifically to track the real visible box under Safari's collapsing/expanding URL and tab bars rather than the static `100vh`, which on iOS undercounts or overcounts depending on chrome state. The code states this directly

— see `crates/git-vista/styles.css`, three call sites (`.app`, the menu inline `max-height`, and the full-screen viewer), each with the comment `iOS 16.4+: track the real visible box under Safari's bars`. Safari (all platforms) shipped `dvh`/`svh`/`lvh` support in 16.4, released March 2023.

Each of those three call sites sets `100vh` first and `100dvh` second (later declarations win), so a browser that doesn't understand `dvh` falls back to the static unit rather than breaking — the floor is a quality floor, not a hard cutoff below which the app fails to render, and `docs/IPAD_DESIGN.md`'s "Browser and PWA Requirements" section separately asks to "test current Safari/iPadOS... do not depend on a WebKit-only feature for correctness," which is consistent with graceful degradation rather than a hard gate.

- **Evidence used:** the in-code comment in `styles.css` and the `dvh`/`vh` fallback pattern itself. `docs/IPAD_DESIGN.md` asks for "modern dynamic viewport units" and "current Safari/iPadOS" testing but does not itself state a version number, so 16.4 is derived from the CSS feature's actual ship date, not asserted from that doc.
- **Evidence not available:** no CI job or manual test matrix currently pins or verifies a minimum Safari version against a real device/simulator — `docs/IPAD_DESIGN.md`'s "Validation Matrix" section lists device/orientation combinations to cover but not a version floor to enforce. If a hard floor (rather than a graceful-degradation target) is wanted, that needs an explicit decision and a device/BrowserStack-class check, neither of which exists today.

Linux kernel: 6.12 or later (Landlock ABI 6)

Why 6.12: the M1.13b sandbox's Strict tier requires Landlock at ABI 6 — `crates/git-vista-server/src/sandbox/mod.rs:175`'s `LANDLOCK_ABI_FLOOR`, six because ABI 6 is the first with the signal and abstract-unix-socket scopes the design uses. Landlock ABI 6 first ships in **Linux 6.12** (November 2024). `main.rs:218` gates every server start on this with no degraded path: a verdict other than `Contained` means no server, full stop — see ADR 0029 for why there is deliberately no fallback tier.

This is a hard requirement, not a recommendation, and it is higher than what current mainstream Linux LTS releases ship:

Distribution	GA kernel	Landlock ABI	Starts?
Ubuntu 22.04 LTS	5.15	1	No
Debian 12 bookworm	6.1	2	No
RHEL 9	5.14	1	No
Ubuntu 24.04 LTS	**6.8**	**4 (measured)**	**No**
Debian 13 trixie	6.12	6	Yes, exactly at the floor
Ubuntu 26.04	7.0	8	Yes

- **Measured, not derived, for the row that matters most:** a stock Ubuntu 24.04 cloud image on titan, asked directly via `syscall(444, NULL, 0, 1)`, reported `kernel=6.8.0-138-generic landlock_abi=4`. On the current Ubuntu LTS — supported by Canonical until 2029 — git-vista's server refuses to start.

- **The remedy:** install an HWE kernel (`sudo apt install linux-generic-hwe-24.04`, which is 6.14 on 24.04.3 and clears the floor) and `bwrap` (`sudo apt install bubblewrap`).
- **Cannot be tested in a container.** A container shares the host kernel, so `ubuntu:22.04` under rootless podman reports the host's Landlock ABI, not the guest's — measured returning 8 (titan's own kernel) rather than anything the 22.04 userspace could claim. There is no way to simulate this floor in CI; the only trustworthy check is the real boot-time probe on the real machine, which is what already runs.
- **Why this floor has no CI-parsed check like the git floor above:** the git floor is a version comparison against a binary CI can cheaply build from source and run against. The kernel floor is a property of the host the server itself runs on — nothing analogous to "build git 2.32 and test against it" exists for a kernel, and per the point above, a container cannot even approximate it. `sandbox::probe::run_at_startup()` already is the single source of truth, enforced live on the real running kernel on every boot — a stronger check than a doc-parsed number could ever be, not a weaker one. See ADR 0111 for the full reasoning, including why this does not reopen ADR 0029.

Signed: thomas2010 · 2026-07-27T20:51:16-04:00

Kernel section added: max · 2026-09-03T20:35:00-04:00